Geth Account 명령어 상세 분석

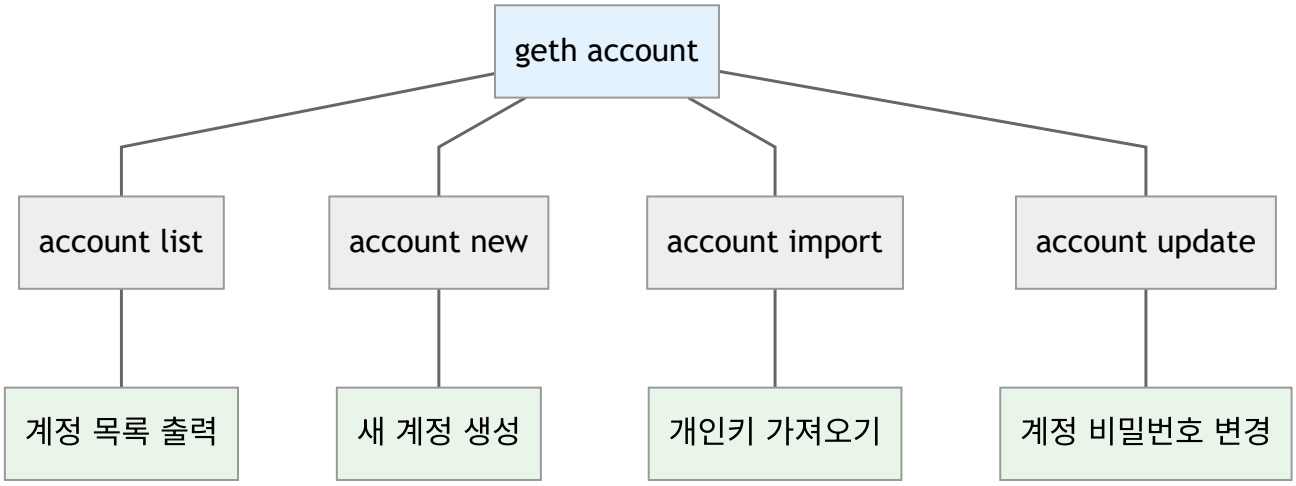
`geth account` 명령어와 그 서브명령어들의 내부 동작 과정을 단계별로 분석한 문서입니다.

> **분석 대상:** `geth account [new|list|import|update]`

> **소스 파일:** `cmd/geth/accountcmd.go`

> **목적:** Ethereum 계정 생성, 관리, 가져오기, 업데이트

🚀 전체 명령어 구조



📋 명령어 정의 구조

메인 Account 명령어

```go
var accountCommand = &cli.Command{
Name: "account",
Usage: "Manage accounts",
```

```
Description: `
Manage accounts, list all existing accounts, import a private key into a new
account, create a new account or update an existing account.

It supports interactive mode, when you are prompted for password as well as
non-interactive mode where passwords are supplied via a given password file.
Non-interactive mode is only meant for scripted use on test networks or known
safe environments.

Keys are stored under <DATADIR>/keystore.
It is safe to transfer the entire directory or the individual keys therein
between ethereum nodes by simply copying.`,

Subcommands: []*cli.Command{
// list, new, import, update 서브명령어들
},
}
```

주요 특징:

- **키스토어 위치**: `<DATADIR>/keystore/` 디렉토리
- **암호화 방식**: Scrypt KDF를 사용한 JSON 키스토어 형식
- **대화형/비대화형 모드**: 패스워드 파일 지원
- **안전한 전송**: 키스토어 디렉토리 전체 복사 가능

🔧 공통 유틸리티 함수들

1. 계정 매니저 생성 (`makeAccountManager()`)

```
// makeAccountManager는 백엔드와 함께 계정 매니저를 생성합니다
func makeAccountManager(ctx *cli.Context) *accounts.Manager {
// 1. 기본 설정 로드
cfg := loadBaseConfig(ctx)

// 2. 빈 계정 매니저 생성
am := accounts.NewManager(nil)

// 3. 키스토어 디렉토리 경로 확인
keydir, isEphemeral, err := cfg.Node.GetKeyStoreDir()
if err != nil {
utils.Fatalf("Failed to get the keystore directory: %v", err)
}
if isEphemeral {
utils.Fatalf("Can't use ephemeral directory as keystore path")
}

// 4. 계정 매니저 백엔드 설정
if err := setAccountManagerBackends(&cfg.Node, am, keydir); err != nil {
```

```
utils.Fatalf("Failed to set account manager backends: %v", err)
}
return am
}
```

동작 상세:

- **설정 로드**: CLI 플래그에서 데이터 디렉토리, 키스토어 경로 등 추출
- **임시 디렉토리 검증**: 메모리 기반 임시 디렉토리 사용 방지
- **백엔드 설정**: 키스토어, 하드웨어 지갑 등 백엔드 초기화

2. 패스워드 파일 읽기 (`readPasswordFromFile()`)

```
// readPasswordFromFile은 주어진 파일의 첫 번째 줄을 읽고,
// 줄 끝을 제거한 후 패스워드와 읽기 성공 여부를 반환합니다.
func readPasswordFromFile(path string) (string, bool) {
if path == "" {
return "", false
}

// 1. 파일 전체 읽기
text, err := os.ReadFile(path)
if err != nil {
utils.Fatalf("Failed to read password file: %v", err)
}
```

```
// 2. 줄 단위로 분할
lines := strings.Split(string(text), "\n")
if len(lines) == 0 {
return "", false
}

// 3. DOS 줄 끝 제거 (Windows 호환성)
return strings.TrimRight(lines[0], "\r"), true
}
```

보안 고려사항:

- 첫 번째 줄만 사용 (추가 데이터 무시)
- 크로스 플랫폼 줄 끝 처리
- 빈 파일 처리

📝 서브명령어 상세 분석

1. `geth account list` - 계정 목록 조회

```
func accountList(ctx *cli.Context) error {
// 1. 계정 매니저 생성
am := makeAccountManager(ctx)
```

```
// 2. 모든 지갑 순회
var index int
for _, wallet := range am.Wallets() {
// 3. 각 지갑의 계정들 순회
for _, account := range wallet.Accounts() {
// 4. 계정 정보 출력
fmt.Printf("Account #%d: {%x} %s\n",
index, account.Address, &account.URL)
index++
}
}
return nil
}
```

실행 흐름:

1. **계정 매니저 초기화**: 키스토어 백엔드 로드
2. **지갑 검색**: 키스토어, 하드웨어 지갑 등 모든 백엔드 검색
3. **계정 열거**: 각 지갑에서 관리하는 계정들 수집
4. **정보 출력**: 주소와 키 파일 경로 표시

출력 예시:

```
$ geth account list
Account #0: {0x742d35cc6634c0532925a3b8d4c9db96c4b4d8b} keystore:///home/user/.
```

```
Account #1: {0x8ba1f109551bd432803012645hac136c0c8b} keystore:///home/user/.eth
```

2. geth account new - 새 계정 생성

```
func accountCreate(ctx *cli.Context) error {
// 1. 기본 설정 로드
cfg := loadBaseConfig(ctx)
keydir, isEphemeral, err := cfg.Node.GetKeyStoreDir()
if err != nil {
utils.Fatalf("Failed to get the keystore directory: %v", err)
}
if isEphemeral {
utils.Fatalf("Can't use ephemeral directory as keystore path")
}

// 2. Scrypt 파라미터 설정
scryptN := keystore.StandardScryptN // 262144 (표준)
scryptP := keystore.StandardScryptP // 1
if cfg.Node.UseLightweightKDF {
scryptN = keystore.LightScryptN // 4096 (경량)
scryptP = keystore.LightScryptP // 6
}
```

```
// 3. 패스워드 획득
password, ok := readPasswordFromFile(ctx.Path(utils.PasswordFileFlag.Name))
if !ok {
password = utils.GetPassPhrase(
"Your new account is locked with a password. Please give a password. Do not forg
true)
}

// 4. 키 생성 및 저장
account, err := keystore.StoreKey(keydir, password, scryptN, scryptP)
if  err != nil {
utils.Fatalf("Failed to create account: %v", err)
}

// 5. 결과 출력
fmt.Printf("\nYour new key was generated\n\n")
fmt.Printf("Public address of the key: %s\n", account.Address.Hex())
fmt.Printf("Path of the secret key file: %s\n\n", account.URL.Path)
fmt.Printf("- You can share your public address with anyone. Others need it to i
fmt.Printf("- You must NEVER share the secret key with anyone! The key controls
fmt.Printf("- You must BACKUP your key file! Without the key, it's impossible to
fmt.Printf("- You must REMEMBER your password! Without the password, it's imposs
return  nil
```

```
}
```

keystore.StoreKey 내부 동작:

```
// accounts/keystore/keystore.go
func StoreKey(dir, password string, scryptN, scryptP int) (accounts.Account,
// 1. 새 개인키 생성
key, err := crypto.GenerateKey()
if err != nil {
return accounts.Account{}, err
}

// 2. 계정 구조체 생성
account := accounts.Account{
Address: crypto.PubkeyToAddress(key.PublicKey),
URL: accounts.URL{Scheme: KeyStoreScheme},
}

// 3. 키 암호화
keyjson, err := EncryptKey(&Key{
Id: uuid.NewRandom(),
Address: account.Address,
PrivateKey: key,
```

```
}, password, scryptN, scryptP)
if err != nil {
return accounts.Account{}, err
}

// 4. 파일명 생성 (UTC 타임스탬프 + 주소)
filename := keyFileName(account.Address)

// 5. 키스토어 파일 저장
return writeKeyFile(filepath.Join(dir, filename), keyjson)
}
```

생성되는 키스토어 파일 형식:

```
{
"address": "742d35cc6634c0532925a3b8d4c9db96c4b4d8b",
"crypto": {
"cipher": "aes-128-ctr",
"ciphertext": "a471054846fb...",
"cipherparams": {
"iv": "6087dab2f9fdbbfaddc31a909735c1e6"
},
"kdf": "scrypt",
"kdfparams": {
```

```
"dklen": 32,
"n": 262144,
"p": 1,
"r": 8,
"salt": "ae3cd4e7013836a3df6bd7241b12db061dbe2c6785853cce422d148a624ce0bd"
},
"mac": "517ead924a9d0dc3124507e3393d175ce3ff7c1e96529c6c555ce9e51205e9b2"
},
"id": "3198bc9c-6672-5ab3-d995-4942343ae5b6",
"version": 3
}
```

3. geth account import - 개인키 가져오기

```
func accountImport(ctx *cli.Context) error {
// 1. 인수 검증
if ctx.Args().Len() != 1 {
utils.Fatalf("keyfile must be given as the only argument")
}
keyfile := ctx.Args().First()

// 2. 개인키 파일 로드
key, err := crypto.LoadECDSA(keyfile)
```

```
if  err != nil {
utils.Fatalf("Failed to load the private key: %v", err)
}

// 3. 계정 매니저 및 키스토어 준비
am := makeAccountManager(ctx)
backends := am.Backends(keystore.KeyStoreType)
if  len(backends) == 0 {
utils.Fatalf("Keystore is not available")
}
ks := backends[0].(*keystore.KeyStore)

// 4. 패스워드 획득
password, ok := readPasswordFromFile(ctx.Path(utils.PasswordFileFlag.Name))
if !ok {
password = utils.GetPassPhrase(
"Your new account is locked with a password. Please give a password. Do not forg
true)
}

// 5. ECDSA 키 가져오기
acct, err := ks.ImportECDSA(key, password)
if  err != nil {
```

```
utils.Fatalf("Could not create the account: %v", err)
}

// 6. 결과 출력
fmt.Printf("Address: {%x}\n", acct.Address)
return nil
}
```

crypto.LoadECDSA 내부 동작:

```
// crypto/crypto.go
func LoadECDSA(file string) (*ecdsa.PrivateKey, error) {
// 1. 파일 읽기
buf, err := os.ReadFile(file)
if err != nil {
return nil, err
}

// 2. 16진수 문자열을 바이트로 변환
key, err := hex.DecodeString(strings.TrimSpace(string(buf)))
if err != nil {
return nil, err
}
```

```
// 3. ECDSA 개인키로 변환
return ToECDSA(key)
}
```

지원하는 개인키 형식:

- **Raw Hex**: `0x1234567890abcdef...` (64자리 16진수)
- **Plain Hex**: `1234567890abcdef...` (0x 접두사 없음)
- **파일 형식**: 텍스트 파일에 16진수 문자열 저장

4. `geth account update` - 계정 업데이트

```
func accountUpdate(ctx *cli.Context) error {
// 1. 인수 검증
if ctx.Args().Len() == 0 {
utils.Fatalf("No accounts specified to update")
}

// 2. 계정 매니저 및 키스토어 준비
am := makeAccountManager(ctx)
backends := am.Backends(keystore.KeyStoreType)
if len(backends) == 0 {
utils.Fatalf("Keystore is not available")
}
```

```
ks := backends[0].(*keystore.KeyStore)


// 3. 각 계정 주소에 대해 업데이트 수행
for _, addr := range ctx.Args().Slice() {
// 4. 주소 형식 검증
if !common.IsHexAddress(addr) {
return errors.New("address must be specified in hexadecimal form")
}
account := accounts.Account{Address: common.HexToAddress(addr)}


// 5. 새 패스워드 입력
newPassword := utils.GetPassPhrase(
"Please give a NEW password. Do not forget this password.", true)


// 6. 업데이트 함수 정의 (재시도 로직 포함)
updateFn := func(attempt int) error {
prompt := fmt.Sprintf("Please provide the OLD password for account %s | Attempt
addr, attempt+1, 3)
password := utils.GetPassPhrase(prompt, false)
return ks.Update(account, password, newPassword)
}


// 7. 최대 3번 시도
```

```
err := updateFn(0)

for attempts := 1; attempts < 3 && errors.Is(err, keystore.ErrDecrypt); attem

err = updateFn(attempts)

}

if err != nil {

return fmt.Errorf("could not update account: %w", err)

}

}

return nil

}
```

ks.Update 내부 동작:

```
// accounts/keystore/keystore.go

func (ks *KeyStore) Update(a accounts.Account, passphrase, newPassphrase string

// 1. 기존 키 파일 로드 및 복호화

key, err := ks.getDecryptedKey(a, passphrase)

if err != nil {

return err

}


// 2. 새 패스워드로 재암호화

keyjson, err := EncryptKey(key, newPassphrase, StandardScryptN, StandardScryptP)

if err != nil {
```

```
return err
}

// 3. 기존 파일 덮어쓰기
return writeKeyFile(a.URL.Path, keyjson)
}
```

🔐 암호화 및 보안

Scrypt KDF 파라미터

| 모드 | N (CPU 비용) | P (병렬성) | 용도 |

| ---------- | ---------- | ---------- | ---------- |

| **Standard** | 262,144 | 1 | 프로덕션 환경 |

| **Light** | 4,096 | 6 | 테스트/개발 환경 |

메모리 사용량 계산:

```
메모리 = 128 * N * r * p 바이트
Standard: 128 * 262144 * 8 * 1 = 268MB
Light: 128 * 4096 * 8 * 6 = 25MB
```

키스토어 파일 구조

암호화 과정:

1. **개인키 생성**: secp256k1 곡선 사용

2. **솔트 생성**: 32바이트 랜덤 솔트
3. **키 파생**: Scrypt로 암호화 키 생성
4. **AES 암호화**: AES-128-CTR 모드로 개인키 암호화
5. **MAC 생성**: 무결성 검증용 HMAC-SHA256

📊 실행 예시 및 로그

새 계정 생성:

```
$ geth account new
Your new account is locked with a password. Please give a password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x742d35Cc6634C0532925a3b8D4C9db96c4b4d8b
Path of the secret key file: /home/user/.ethereum/keystore/UTC--2023-01-01

- You can share your public address with anyone. Others need it to
- You must NEVER share the secret key with anyone! The key controls
- You must BACKUP your key file! Without the key, it's impossible to a
- You must REMEMBER your password! Without the password, it's impossible to d
```

계정 목록 조회:

```
$ geth account list
Account #0: {0x742d35cc6634c0532925a3b8d4c9db96c4b4d8b} keystore:///home/user/.
Account #1: {0x8ba1f109551bd432803012645hac136c0c8b} keystore:///home/user/.eth
```

개인키 가져오기:

```
$ geth account import private.key
Your new account is locked with a password. Please give a password.
Password:
Address: {0x9876543210fedcba0987654321fedcba09876543}
```

🛠️ 문제 해결

일반적인 오류들

1. 키스토어 디렉토리 권한 오류:

```
Fatal: Failed to get the keystore directory: mkdir /root/.ethereum: per
```

해결: `--datadir` 또는 `--keystore` 플래그로 쓰기 가능한 디렉토리 지정

2. 잘못된 개인키 형식:

```
Fatal: Failed to load the private key: invalid hex string
```

해결: 개인키 파일이 64자리 16진수 문자열인지 확인

3. 패스워드 불일치:

```
could not decrypt key with given password
```

해결: 올바른 패스워드 입력 또는 키 파일 복구

고급 사용법

1. 비대화형 모드 (스크립트용):

```
echo "mypassword" > password.txt
geth account new --password password.txt
```

2. 커스텀 키스토어 디렉토리:

```
geth account list --keystore ./custom-keystore
```

3. 경량 KDF 사용 (빠른 테스트용):

```
geth account new --lightkdf
```

📁 파일 시스템 구조

키스토어 디렉토리:

```
~/.ethereum/keystore/
├── UTC--2023-01-01T12-00-00.000000000Z--742d35cc6634c0532925a3b8d4c9db96c4b4d8b
├── UTC--2023-01-02T12-00-00.000000000Z--8ba1f109551bd432803012645hac136c0c8b
└── UTC--2023-01-03T12-00-00.000000000Z--9876543210fedcba0987654321fedcba0987654
```

파일명 형식:

- `UTC--<ISO8601 타임스탬프>--<주소>`
- 타임스탬프는 키 생성 시각
- 주소는 체크섬 없는 소문자 16진수

참고 자료:

- Ethereum 키스토어 형식
- Scrypt KDF 사양
- secp256k1 곡선

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.